# Prática — Criptografia Simétrica e Modos de Operação

DCC075 – Segurança em Sistemas Computacionais · UFJF · Lucas Gonçalves Rocha · Matrícula 202265187C · Individual · Outubro de 2026

---

## Parte 1 — Modos de operação do AES-128 com OpenSSL

O roteiro, escrito para Ubuntu, foi executado no macOS: OpenSSL 3.5.9 instalado pelo Homebrew (o `openssl` padrão do sistema é o LibreSSL 3.3.6), ImageMagick 7.1.2 (`magick` no lugar de `convert`) e `curl` no lugar de `wget`. Todos os passos estão reunidos no script `tux.sh`, no repositório da disciplina, que regenera do zero a chave, os IVs e as imagens cifradas.

### 1. Ambiente

```
lucas@MacBook-Air-de-Lucas 06-modos-operacao % which openssl
openssl version
magick -version | head -1
/opt/homebrew/opt/openssl@3/bin/openssl
OpenSSL 3.5.9 29 Sep 2026 (Library: OpenSSL 3.5.9 29 Sep 2026)
Version: ImageMagick 7.1.2-34 Q16-HDRI aarch64 c5d4ca8a0:20261014 https://imagemagick.org
```

Figura 1 — Versões do OpenSSL e do ImageMagick utilizadas.

### 2. Geração da chave com PBKDF2

```
lucas@MacBook-Air-de-Lucas 06-modos-operacao % openssl enc -pbkdf2 -aes-128-ecb -k minha-senha-secreta -P
openssl enc -pbkdf2 -aes-128-ecb -k minha-senha-secreta -P
openssl enc -pbkdf2 -aes-128-ecb -k minha-senha-secreta -P | awk -F= '/^key/{print $2}' > key.priv
cat key.priv
git check-ignore -v key.priv
salt=383949C9187DA891
key=C35F857BBE9E0F1CE3A101CE60D5CFDC
salt=527DF1DAFB800654
key=3CD99C74C652D979F7527B9B676BB320
3B35C3FCED442147F9EBBDCF1CD98F57
openssl/06-modos-operacao/.gitignore:1:key.priv key.priv
```

Figura 2 — Derivação da chave AES-128 a partir de uma senha.

O comando `openssl enc -pbkdf2 -P` deriva uma chave de 128 bits a partir da senha com o PBKDF2 (HMAC-SHA-256 aplicado iterativamente sobre a senha e um *salt*) e apenas imprime o salt e a chave, sem cifrar nada. As duas execuções com a mesma senha produziram salts e chaves diferentes: o salt aleatório torna cada derivação única, o que inviabiliza tabelas pré-computadas, e as iterações encarecem cada tentativa de força bruta. A chave foi salva em `key.priv` (32 dígitos hexadecimais = 128 bits), e o `git check-ignore` confirma que o arquivo está fora do repositório.

Nos passos seguintes, a chave é passada crua com `-K`. Com `-k` (senha), o OpenSSL derivaria a chave e gravaria um cabeçalho `Salted__` de 16 bytes no início do arquivo cifrado, o que desalinharia os pixels da imagem. Vale notar que passar a senha na linha de comando a expõe no histórico do shell e na lista de processos (CWE-214); fora do laboratório, o adequado seria `-pass env:VAR` ou a digitação interativa.

### 3. Cifragem de texto em ECB

```
lucas@MacBook-Air-de-Lucas 06-modos-operacao % head -3 cleartext.txt
openssl enc -aes-128-ecb -in cleartext.txt -K $(cat key.priv) -out ciphertext.txt
xxd ciphertext.txt | head -5
ls -l cleartext.txt ciphertext.txt
openssl enc -d -aes-128-ecb -in ciphertext.txt -K $(cat key.priv) | head -3
diff cleartext.txt <(openssl enc -d -aes-128-ecb -in ciphertext.txt -K $(cat key.priv)) && echo "IDENTICO"
This is our world now... the world of the electron and the switch, the
beauty of the baud.  We make use of a service already existing without paying
for what could be dirt-cheap if it wasn't run by profiteering gluttons, and
00000000: 5d36 7b97 6046 1d97 1cc4 6c1c 0462 708c  ]6{.`F....l..bp.
00000010: 04a7 f218 3dd8 6708 5ae0 47aa aeb5 f908  ....=.g.Z.G.....
00000020: dcc9 c46d a717 f813 991f b430 3363 ef22  ...m.......03c."
00000030: 0f91 a310 ac92 7a8d 4b25 6045 d43e 2605  ......z.K%`E.>&.
00000040: 0f3c d8ac 3e86 3a64 4f71 ebde 445d fb57  .<..>.:dOq..D].W
-rw-r--r--@ 1 lucas  staff  976 20 out 22:18 ciphertext.txt
-rw-r--r--  1 lucas  staff  971 20 out 22:14 cleartext.txt
This is our world now... the world of the electron and the switch, the
beauty of the baud.  We make use of a service already existing without paying
for what could be dirt-cheap if it wasn't run by profiteering gluttons, and
IDENTICO
```

Figura 3 — Cifragem e decifragem do trecho do *Hacker Manifesto* em AES-128-ECB.

O trecho do *Hacker Manifesto* (971 bytes) cifrado em AES-128-ECB tornou-se uma sequência de bytes sem significado, como mostra a saída do `xxd`. O texto cifrado tem 976 bytes porque o ECB só processa blocos completos de 16 bytes: o *padding* PKCS#7 completou o último bloco (971 = 60 × 16 + 11) com 5 bytes de valor `0x05`, totalizando 61 blocos. A decifragem com a mesma chave (`-d`) recuperou o texto, e o `diff` comprovou que ele é idêntico ao original, byte a byte.

### 4. Preparo da imagem

```
lucas@MacBook-Air-de-Lucas 06-modos-operacao % curl -sLO https://upload.wikimedia.org/wikipedia/commons/a/af/Tux.png
file Tux.png
magick Tux.png tux.ppm
head -n 3 tux.ppm > header.txt
tail -n +4 tux.ppm > body.bin
cat header.txt
wc -c tux.ppm header.txt body.bin
Tux.png: PNG image data, 265 x 314, 8-bit colormap, non-interlaced
P6
265 314
255
  249645 tux.ppm
      15 header.txt
  249630 body.bin
  499290 total
```

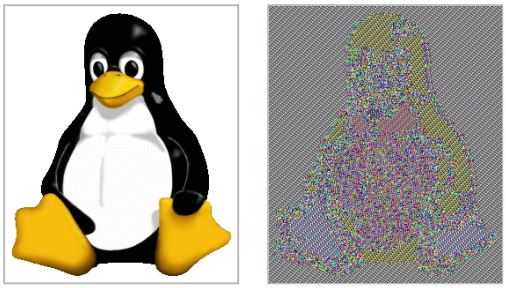
Figura 4 — Conversão do Tux para PPM e separação entre cabeçalho e corpo.

O Tux foi convertido de PNG para PPM (tipo P6). O PNG é comprimido, então cifrá-lo diretamente produziria apenas um arquivo corrompido, sem padrão visível. O PPM guarda os pixels crus (3 bytes RGB por pixel) depois de um cabeçalho de texto de 3 linhas — `P6`, as dimensões (265 × 314) e o valor máximo por canal (255) —, que aqui ocupa 15 bytes. O corpo tem 265 × 314 × 3 = 249.630 bytes, e 15 + 249.630 = 249.645, exatamente o tamanho do `tux.ppm`, o que confirma o corte. Só o corpo é cifrado; o cabeçalho volta em claro, para que o visualizador consiga abrir a imagem e desenhar o texto cifrado como se fossem pixels.

## 5. Imagem em ECB

```
lucas@MacBook-Air-de-Lucas 06-modos-operacao % openssl enc -aes-128-ecb -in body.bin -K $(cat key.priv) -out body.ecb.en
c
cat header.txt body.ecb.enc > tux-ecb-enc.ppm
magick tux-ecb-enc.ppm imagens/tux-ecb.png
wc -c body.bin body.ecb.enc
xxd -p -c 16 body.ecb.enc | sort | uniq -c | sort -rn | head -5
  249630 body.bin
  249632 body.ecb.enc
  499262 total
7246 3cf3f2a0197b0d7735c6dc616351b2d6
 524 a44819563673447bee0c0a71f1aa20b2
 512 ea3aa45c97c7e06316453a6dea25bc24
 505 e12c24d4e434552158b5a258caf75560
 410 5a26affa4957bc05b094c43c09a451b6
```

Figura 5 — Cifragem do corpo em ECB e contagem dos blocos cifrados mais repetidos.



Original AES-128-ECB

Figura 6 — O Tux original e o resultado da cifragem em ECB: a silhueta continua reconhecível.

No ECB, cada bloco é cifrado de forma independente ($c_i = E_k(m_i)$); blocos de texto plano iguais geram blocos cifrados iguais. Por isso a imagem cifrada mantém a silhueta do Tux. A contagem de blocos mostra o motivo: dos 15.602 blocos, um mesmo bloco cifrado aparece 7.246 vezes (46%) — é o fundo branco —, e 242 blocos distintos se repetem. Outros detalhes do resultado:

- O fundo aparece em listras diagonais porque cada linha da imagem tem 795 bytes e 795 mod 16 = 11: o mesmo bloco cifrado começa deslocado a cada linha.
- A barriga, que no original tem degradê, virou ruído, porque pixels vizinhos diferentes não formam blocos repetidos. O ECB não “protegeu” essa região: ali simplesmente não havia repetição para vazar. Uma cifra cuja segurança depende do conteúdo cifrado é inaceitável.
- Em execuções com chaves diferentes, os valores dos blocos cifrados mudaram, mas as contagens (7.246, 524, 512, 505 e 410) foram idênticas. O vazamento é uma propriedade do modo aplicado a dados com repetição, e nenhuma chave, por mais forte que seja, o corrige.

O tamanho passou de 249.630 para 249.632 bytes (2 bytes de padding), que o visualizador ignora.

## 6. Imagem em CBC

```
lucas@MacBook-Air-de-Lucas 06-modos-operacao % openssl rand -hex 16 > iv.txt
cat iv.txt
openssl enc -aes-128-cbc -in body.bin -K $(cat key.priv) -iv $(cat iv.txt) -out body.cbc.enc
cat header.txt body.cbc.enc > tux-cbc-enc.ppm
magick tux-cbc-enc.ppm imagens/tux-cbc.png
wc -c body.bin body.cbc.enc
xxd -p -c 16 body.cbc.enc | sort | uniq -c | sort -rn | head -5
48d92ed241043003ab0b2b8b3f7487ce
  249630 body.bin
  249632 body.cbc.enc
  499262 total
    1 fffe26bfb81bf8b56c7f9f61553a2aca
    1 fff73e58d049611513ef6bc7eb4aded4
    1 ffec8d81ffb08c5ccdb9926eb330d311
    1 ffe71c432c829f9942f0bc03b10132be
    1 ffe14ef4fd960d4c680ebdde0a7dd493
```

Figura 7 — Geração do IV, cifragem em CBC e contagem de blocos repetidos.

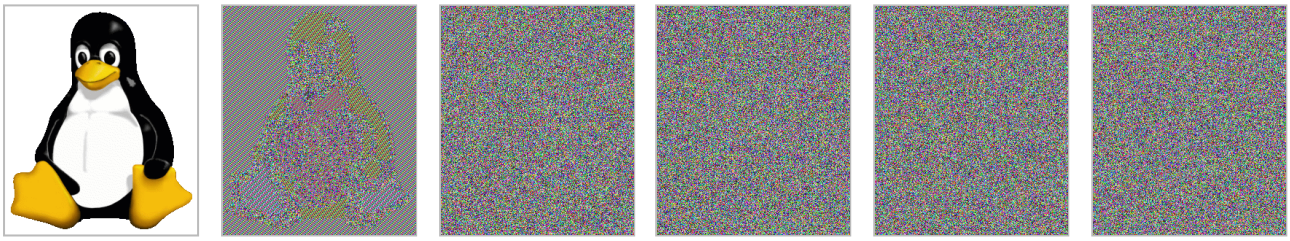
No CBC, cada bloco é combinado por XOR com o bloco cifrado anterior antes de passar pelo AES ($c_i = E_k(m_i \oplus c_{i-1})$), e o primeiro bloco usa um vetor de inicialização (IV) aleatório de 128 bits gerado com `openssl rand`. Blocos de pixels idênticos entram no AES com valores diferentes, e a imagem cifrada é ruído uniforme, sem vestígio da silhueta (Figura 9). Nenhum bloco cifrado se repete: todas as contagens são 1. O IV não precisa ser secreto, mas deve ser imprevisível e nunca reutilizado com a mesma chave. Assim como o ECB, o CBC usa padding (249.632 bytes).

A compressão oferece uma evidência adicional: o PNG gerado a partir da imagem em ECB ocupa 82 KB, porque ainda há estrutura a comprimir, enquanto o da imagem em CBC ocupa 250 KB, mais que os próprios pixels. Um bom texto cifrado é indistinguível de ruído e, portanto, incompressível.

### 7. Imagens em CFB, OFB e CTR

```
lucas@MacBook-Air-de-Lucas 06-modos-operacao % for modo in cfb ofb ctr; do
  openssl rand -hex 16 > iv-$modo.txt
  openssl enc -aes-128-$modo -in body.bin -K $(cat key.priv) -iv $(cat iv-$modo.txt) -out body.$modo.enc
  cat header.txt body.$modo.enc > tux-$modo-enc.ppm
  magick tux-$modo-enc.ppm imagens/tux-$modo.png
done
wc -c body.bin body.*.enc
[for modo in ecb cbc cfb ofb ctr; do echo "$modo: $(xxd -p -c 16 body.$modo.enc | sort | uniq -d | wc -l) blocos distinto]
s que se repetem"; done
  249630 body.bin
  249632 body.cbc.enc
  249630 body.cfb.enc
  249630 body.ctr.enc
  249632 body.ecb.enc
  249630 body.ofb.enc
 1497784 total
ecb:      242 blocos distintos que se repetem
cbc:        0 blocos distintos que se repetem
cfb:        0 blocos distintos que se repetem
ofb:        0 blocos distintos que se repetem
ctr:        0 blocos distintos que se repetem
```

Figura 8 — Cifragem nos modos CFB, OFB e CTR (IV próprio para cada um) e comparação entre os cinco modos.



Original ECB CBC CFB OFB CTR

Figura 9 — Resultado visual dos cinco modos com o mesmo AES-128 e a mesma chave.

Os três modos transformam o AES em uma cifra de fluxo: geram um *keystream* e fazem XOR com os dados. No CFB, o keystream é o AES do bloco cifrado anterior ($c_i = m_i \oplus E_k(c_{i-1})$); no OFB, é o AES aplicado repetidamente ao IV, independentemente da mensagem; no CTR, é o AES de um contador ($c_i = m_i \oplus E_k(IV + i)$), o que permite cifrar blocos em paralelo e é a base do AES-GCM. Como o XOR atua byte a byte, não há padding: os arquivos cifrados têm exatamente 249.630 bytes. As três imagens são ruído e nenhum bloco se repete. Cada modo recebeu um IV próprio porque, nos modos de fluxo, repetir o par chave/IV repete o keystream, e então $c_1 \oplus c_2 = m_1 \oplus m_2$ (*two-time pad*).

### 8. Comparação e conclusão

| Modo | Usa IV | Padding | Tamanho cifrado (bytes) | Blocos distintos repetidos | Silhueta visível |
|---|---|---|---|---|---|
| ECB | não | sim | 249.632 | 242 (o mais frequente: 7.246×) | **sim** |
| CBC | sim | sim | 249.632 | 0 | não |
| CFB | sim | não | 249.630 | 0 | não |
| OFB | sim | não | 249.630 | 0 | não |
| CTR | sim | não | 249.630 | 0 | não |

O experimento mostra que a segurança de uma cifra de bloco depende tanto do modo de operação quanto do algoritmo: com o mesmo AES-128 e a mesma chave, o ECB expõe a estrutura da imagem, enquanto CBC, CFB, OFB e CTR a escondem por completo. O ECB não deve ser usado para dados com mais de um bloco. Os demais modos garantem apenas confidencialidade — não detectam alterações no texto cifrado —, e por isso protocolos atuais, como o TLS 1.3, usam modos autenticados (AEAD), como AES-GCM e ChaCha20-Poly1305, que o `openssl enc` não oferece.

## Parte 2 — O ataque Sweet32

**[ESCREVER] Parágrafo 1 — Visão geral**

O que é o ataque, quem o publicou e quando, quais cifras (Triple-DES, Blowfish) e protocolos (HTTPS/TLS, OpenVPN) afeta, CVE-2016-2183 e CVE-2016-6329.

**[ESCREVER] Parágrafo 2 — Explicação**

Bloco de 64 bits e paradoxo do aniversário (~$2^{32}$ blocos ≈ 32 GB com a mesma chave); colisão no CBC revelando o XOR de dois blocos de texto plano; cenário do cookie de sessão com JavaScript gerando tráfego; por que o tamanho da chave não importa; mitigações.

**[ESCREVER] Parágrafo 3 — Conclusão e visão crítica**

Sua opinião: o ataque é prático (centenas de GB, cerca de dois dias)? Por que cifras de 64 bits continuavam habilitadas em 2016? O que o caso ensina sobre compatibilidade com legado e configurações padrão?

### Referências


[1] S. Prado. *A hands-on approach to symmetric-key encryption*. sergioprado.blog (base do roteiro da prática).

[2] OpenSSL Project. *openssl-enc — symmetric cipher routines*. Documentação do OpenSSL.

[3] M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*. NIST SP 800-38A, 2001.

[4] K. Moriarty, B. Kaliski, A. Rusch. *PKCS #5: Password-Based Cryptography Specification Version 2.1*. RFC 8018, 2017.

[5] K. Bhargavan, G. Leurent. *On the Practical (In-)Security of 64-bit Block Ciphers: Collision Attacks on HTTP over TLS and OpenVPN*. ACM CCS 2016. https://sweet32.info/